DOCUMENTACIÓN - Parte 3 (Entrega Final)

Caso #: 3 Urbana Cult

Empresa de Desarrollo: Sintaxia - Grupo 19

Integrantes: Diego Rodríguez, Brian Lavandera, Sergio Vargas, Cristian Villagra

ÍNDICE

1. Enlaces de Entrega (Despliegue, Repositorio, Drive y Video)
2. Fuentes y Bibliografía Utilizadas
3. Introducción y Alcance de la Entrega Final
4. Matriz de Tecnologías (Evolución Entregas 1, 2 y 3)
5. Integrantes, Roles y Responsabilidades
6. Descripción Funcional y Matriz de Permisos por Rol
7. Módulos del Proyecto y Justificación de Decisiones Técnicas
8. Persistencia de Datos en MongoDB Atlas y Diagramas (DER y Secuencia)
9. Arquitectura MVC, Middlewares y Procesamiento de Peticiones
10. Trazabilidad de Requerimientos (RF / RNF) y Documentación de Pruebas
11. Mantenimiento, Evolución y Uso Consciente de Inteligencia Artificial
12. Conclusión Final del Cuatrimestre

1. Enlaces de Entrega

- **Aplicación Desplegada en Línea (Render - Producción):**
  https://urbana-cult.onrender.com

- **Repositorio GitHub (3° Entrega Final):**
  https://github.com/brianlavandera1/tp2

- **Carpeta de Google Drive (Documentación y entregas previas/actual):**
  https://drive.google.com/drive/folders/1FJnjrQ9gMjxc1aAcUBWMUMIkWc-9bSrL?usp=drive_link

- **Trazabilidad Histórica (Entregas 1 y 2):**
  Repositorio inicial: https://github.com/Crrisst/TP1-BackEnd/ · Video Parte 1 · Video Parte 2

- **Link al Video de Defensa (Coloquio grupal - Entrega Final):**
  [Insertar enlace de Google Drive / YouTube del video final antes de subir al Campus]

2. Fuentes y Bibliografía Utilizadas

Material oficial de la cátedra y entregas previas: Mapa de Contenidos de *Desarrollo Web Backend* (**Bloques 1 al 11.2**) y cuadernillos teóricos **Módulos M1 a M13** del Aula Virtual (`aulasvirtuales.bue.edu.ar`): *Introducción a Node.js y Express* (Bloques 1–2), *Patrón MVC, POO y Persistencia con FS* (Bloques 3–4), *Middlewares y Motor de Plantillas Pug* (Bloques 5.1–5.2), *Asincronía (Callbacks, Promises, async/await), HTTP y Estados* (Bloques 6.1–6.3), *MongoDB, Mongoose y Operadores de Consulta* (Bloques 7.1–7.3), *Seguridad, Modelo RBAC y Hashing* (Bloque 8.0), *APIs RESTful, MongoDB Atlas y Despliegue en Render* (Bloque 9.0), y *Autenticación con Sesiones, Cookies y JSON Web Tokens — RFC 7519* (Bloques 10.0–10.2). Asimismo, se tomaron como base los artefactos de *Ingeniería de Software* para el Caso #3 (Urbana Cult) y las entregas previas *DOCUMENTACIÓN - Parte 1* y *DOCUMENTACIÓN - Parte 2* del equipo Sintaxia.

Documentación técnica oficial consultada: Node.js v20+ Documentation (nodejs.org/docs); Express 5 API Reference (expressjs.com); Mongoose ODM v9 Docs — Schemas, Populate, Plugins & Transactions (mongoosejs.com/docs); MongoDB Manual — Query & Projection Operators y Atlas Cloud (mongodb.com/docs); RFC 7519 JSON Web Token & librería `jsonwebtoken` (jwt.io); OWASP Session Management & CSRF Prevention Cheat Sheets (cheatsheetseries.owasp.org); Helmet.js Security Headers (helmetjs.github.io); Pug Template Engine (pugjs.org); Render Web Services Deployment Guide (render.com/docs).

3. Introducción y Alcance de la Entrega Final

En esta tercera instancia final de la materia *Desarrollo Web Backend*, la consultora estudiantil **Sintaxia (Grupo 19)** consolidó la versión productiva de **Urbana Cult**, un sistema integral para la gestión de salas culturales, programación de espectáculos, membresías de beneficios, boletería presencial, venta online de entradas y validación de accesos mediante códigos QR.

Cumpliendo íntegramente la hoja de ruta planteada al cierre de la segunda entrega, el proyecto evolucionó desde un Producto Mínimo Viable local con sesiones en memoria hacia una arquitectura robusta desplegada en la nube (**Render + MongoDB Atlas**). En esta etapa se incorporó encriptación irreversible de contraseñas con `bcrypt`, persistencia de sesiones en base de datos con `connect-mongo`, una API JSON protegida mediante **JSON Web Tokens (JWT)** firmados digitalmente, control de acceso granular de cuatro roles operativos más protección de Superadministrador, transacciones atómicas para evitar sobreventa de localidades, borrado lógico auditable en todas las colecciones maestras, bitácora de actividad de usuarios y exportación administrativa a planillas Excel.

Alcance definido: El sistema cubre el ciclo comercial completo de un centro cultural: descubrimiento público de la cartelera con filtros sin recarga, registro validado en tres capas, contratación de planes de membresía (Estándar, Premium y VIP), carrito y cobro online simulado, venta física en ventanilla con ticket imprimible, acreditación en puerta por lectura de QR, cancelación con restitución automática de cupos y tableros gerenciales de recaudación neta. Quedan fuera del alcance la integración con pasarelas bancarias reales y la numeración individual de butacas por plano gráfico.

4. Matriz de Tecnologías (Evolución Entregas 1, 2 y 3)

La siguiente matriz resume la progresión tecnológica real del proyecto a lo largo de las tres instancias evaluativas de la cursada:

Eje / Capa Técnica	1° Entrega (MVC + POO + FS)	2° Entrega (PMV + MongoDB)	3° Entrega (Cloud, RBAC & JWT)
Servidor y Arquitectura	Node.js + Express, patrón **MVC** inicial, **POO** en dominio (herencia `Cliente` / `Administrador`), logger HTTP y middleware global de errores.	MVC consolidado (`routes` , `controllers` , `models` , `views`) y programación asincrónica con `async/await` .	Express 5 modularizado con `config/` , `middlewares/` (11 módulos especializados) y `utils/` reutilizables.
Persistencia de Datos	Archivos planos JSON en `/data/` (`usuarios.json` , `eventos.json` , `salas.json`) mediante módulo nativo `fs` .	MongoDB local + ODM Mongoose (4 esquemas base: `Usuario` , `Sala` , `Evento` , `Entrada`).	**MongoDB Atlas** (Cloud) + respaldo local, **8 esquemas**, transacciones, operadores (`$gte` , `$inc` , `$or` , `$elemMatch`), agregaciones y **borrado lógico**.

Eje / Capa Técnica	1° Entrega (MVC + POO + FS)	2° Entrega (PMV + MongoDB)	3° Entrega (Cloud, RBAC & JWT)
Autenticación y Sesiones	Sin sesiones persistentes; distinción conceptual de cuentas mediante clases POO y parámetros.	`express-session` en memoria RAM del proceso; contraseñas en texto plano con fines académicos.	Híbrida: **Sesiones en MongoDB** (`connect-mongo` , cookies `HttpOnly` / `Secure`) + **JWT (`jsonwebtoken` HS256)** en API REST. Hash con **`bcrypt`** .
Autorización y Seguridad	Validaciones de datos obligatorios, formato y tipos en controladores antes de persistir en JSON.	2 roles básicos (`Cliente` y `Admin`) con middlewares por ruta y expresiones regulares en formularios.	Modelo **RBAC de 4 roles + Superadmin**, `helmet` (CSP), anti-CSRF (`verificarOrigen`), anti-fuerza bruta (`limiteLogin` HTTP 429) y triple validación.
Vistas y Experiencia (Front)	Motor de plantillas **Pug** inicial (layouts e iteraciones dinámicas) + respuestas JSON para API CRUD.	Catálogo público en Pug, carrito de compras en sesión, pasarela simulada y ticket con QR en cliente.	Diseño integral *Urbana Cult* responsive mobile, filtros dinámicos, fotos binarias de artistas (`multer`), membresías, boletería física y lector QR en puerta.
Pruebas, Módulos y Despliegue	Ejecución local (`localhost`) y pruebas de endpoints CRUD verificadas con **Thunder Client**.	Variables de entorno con `dotenv` y pruebas funcionales integrales de flujo web.	Despliegue productivo en **Render** (HTTPS + `trust proxy`), reportes `exceljs` , `qrcode` SVG y bitácora de auditoría de actividad.

5. Integrantes, Roles y Responsabilidades

Integrante	Rol en el Equipo	Evolución y Responsabilidades en la Entrega Final
Brian Lavandera	Líder Técnico / Seguridad & Cloud	*(E1: POO en Usuarios · E2: Auth y Sesiones).* En esta entrega final lideró la configuración de entorno (`config.js`), la conexión resiliente a MongoDB Atlas (`db.js`), la autenticación dual con sesiones persistentes (`connect-mongo`) y emisión/verificación de **JWT** (`jwt.middleware.js` , `POST /auth/token`), el hash con `bcrypt` , las defensas `helmet` / CSRF / rate-limit y el despliegue productivo en **Render**.
Diego Rodríguez	Backend Developer / Post-Venta & Auditoría	*(E1: Update Salas/Eventos · E2: Modelo Entrada y QR).* Evolucionó `Entrada.js` incorporando cancelación transaccional con restitución de stock y registro en `CancelacionLog.js` , desarrolló la validación atómica de QR en puerta (`/admin/validar/:codigo`), el plugin transversal de borrado lógico (`borradoLogico.js`), la bitácora de actividad por sesión (`Actividad.js`) y la documentación técnica.

Integrante	Rol en el Equipo	Evolución y Responsabilidades en la Entrega Final
Sergio Vargas	Full-Stack Developer / UX Cliente & Carrito	*(E1: Validaciones y Thunder Client · E2: Carrito y Checkout).* Desarrolló la interfaz responsive en Pug (`estilosUrbana.pug`), la cartelera con filtros combinados sin recarga, la carga validada de fotografías de artistas con `multer` (`Entidad.js`), el sistema de membresías con descuentos automáticos (Estándar, Premium 15 %, VIP 30 %) y el checkout atómico por sector.
Cristian Villagra	Backend Developer / Back-Office, API & Reportes	*(E1: Arquitectura MVC y Pug base · E2: Panel Admin y Salas).* Consolidó el Back-Office (`/admin`) bajo modelo RBAC para Administrador, Supervisor y Lector, implementó el módulo de boletería presencial en sucursal (`boleteria.pug`), los pipelines de agregación financiera en MongoDB, la exportación de reportes a `.xlsx` (`excelController.js`) y los endpoints de la API RESTful `/entradas` .

6. Descripción Funcional y Matriz de Permisos por Rol

Portal Público y Experiencia del Cliente (B2C): Cualquier visitante puede explorar la portada, la cartelera cultural con precios por sector (General, Platea y VIP), la calculadora de ahorro de membresías y el centro de ayuda sin necesidad de iniciar sesión. Al autenticarse, el cliente accede a su carrito persistente, aplica automáticamente el descuento de su membresía en el checkout, descarga e imprime sus entradas con código QR único, gestiona sus datos personales y puede cancelar tickets vigentes devolviendo el cupo al espectáculo.

Panel de Gestión Operativa y Auditoría (Back-Office): El personal interno ingresa a un panel lateral adaptado estrictamente a su jerarquía organizacional. De este modo —tal como se analizó en clase respecto a la seguridad interna en empresas—, un empleado con rol de lectura o supervisión operativa no puede alterar usuarios privilegiados ni acceder a funciones fuera de su competencia, validándose cada permiso en el servidor mediante middlewares antes de ejecutar cualquier controlador.

Funcionalidad del Sistema	Cliente	Lector	Supervisor	Admin
Explorar cartelera, comprar online, gestionar membresía, perfil y cancelar entradas propias	✅	—	—	—
Consultar dashboard, listado de salas/eventos/artistas, pagos, cancelaciones e informes (y Excel)	—	✅	✅	✅
Crear, editar y dar de baja lógica salas, eventos y artistas/entidades (con fotografía)	—	—	✅	✅
Operar Boletería Física en sucursal y validar entradas con código QR en puerta	—	—	✅	✅
Auditar el registro detallado de actividad por sesión, clics e histórico de eliminados	—	—	✅	✅
Gestionar cuentas de usuarios y personal interno (solo el Superadmin `admin` gestiona a otros Admins)	—	—	—	✅
Operaciones de escritura en la API JSON (`POST` , `PUT` , `DELETE /entradas`) con JWT	—	—	—	✅

7. Módulos del Proyecto y Justificación de Decisiones Técnicas

- ✓ **Autenticación Dual (Sesiones + JWT) y Seguridad:** Login web con regeneración de ID de sesión (anti *session fixation*), cookie `httpOnly` / `sameSite=lax` / `secure`, y endpoint `POST /auth/token` para clientes API mediante tokens firmados con `HS256`. Incluye bloqueo temporal ante 5 intentos fallidos en 15 minutos (`limiteLogin`).
- ✓ **Gestión de Salas, Artistas y Eventos:** Verificación matemática de aforos por sector (`General + Platea + VIP = Capacidad`), bloqueo de modificación estructural si existen funciones asociadas (HTTP 409) y carga validada por *magic bytes* de imágenes JPG/PNG/WEBP de hasta 2 MB en MongoDB.
- ✓ **Carrito, Membresías y Transacciones Atómicas:** Aplicación automática de descuentos (0 %, 15 %, 30 %), validación de medios de pago y ejecución envuelta en `conTransaccion()` (`utils/transaccion.js`): usa sesiones ACID nativas en MongoDB Atlas y compensación automática si se ejecuta sobre un MongoDB local *standalone*.
- ✓ **Boletería Presencial y Control de Acceso QR:** Venta en ventanilla con descuento en efectivo, generación de QR en formato SVG vectorial (`qrcode`) apuntando a `/admin/validar/:codigo` y marcado atómico `Completado → Usado` que impide el reingreso con el mismo ticket.
- ✓ **Auditoría Contable, Borrado Lógico y Exportación Excel:** Ninguna entidad maestra se destruye físicamente (plugin `borradoLogico.js`). Los informes financieros descuentan automáticamente las entradas canceladas y permiten descargar libros `.xlsx` formateados mediante `exceljs`.

Justificación de decisiones arquitectónicas, herramientas omitidas y módulos adicionales

Decisiones alineadas a los bloques teóricos de la cátedra: Para el control de permisos se adoptó el modelo **RBAC** (*Role-Based Access Control*, Bloques 8.0 y 10.0) restringiendo roles con `enum` en Mongoose y validándolos en servidor mediante `verificarRol`. Para el resguardo de credenciales, se evolucionó del concepto de hash con *salt* visto con `PBKDF2` (Bloque 8.0) hacia `bcrypt` con factor de costo 10 (Bloque 10.2), que gestiona el *salt* criptográfico por usuario de forma nativa y resistente a fuerza bruta. En materia de comunicación HTTP y consultas (Bloques 6.2, 6.3, 7.2 y 7.3), las operaciones críticas emplean operadores atómicos y lógicos de MongoDB (`$gte`, `$lte`, `$or`, `$ne`, `$inc`, `$elemMatch`, `$expr`) devolviendo códigos HTTP semánticos estrictos (`200`, `201`, `302`, `400`, `401`, `403`, `404`, `409`, `429`, `500`).

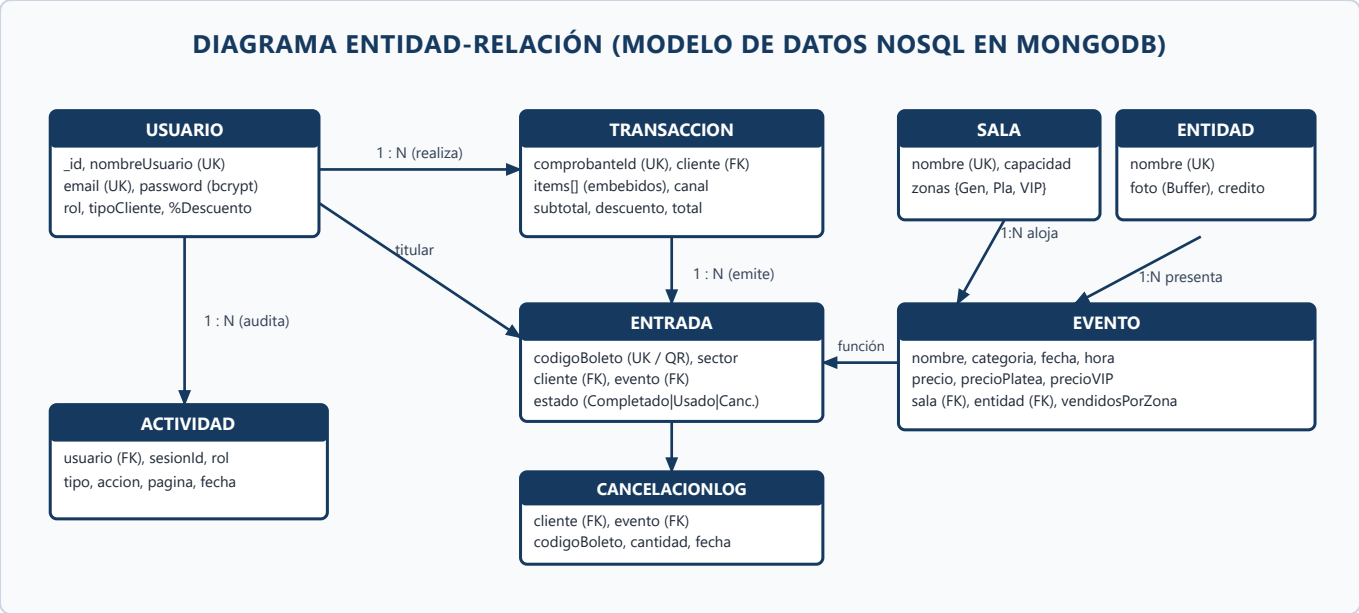
Herramientas no utilizadas: No se incorporó **Passport.js** dado que el sistema emplea credenciales locales sin proveedores OAuth externos, permitiendo control directo sobre la regeneración de sesión, el bloqueo por intentos y la firma de JWT. Tampoco se habilitó el middleware `cors` abierto (Bloque 6.2) porque las vistas Pug se sirven desde el mismo origen del backend, reforzándose en cambio con validación estricta de cabecera `Origin` / `Referer` (`verificarOrigen`) contra ataques CSRF; ni se utilizó `mongoose-unique-validator` por encontrarse deprecado en Mongoose 8/9, resolviendo la unicidad mediante índices nativos `unique: true` y captura del código `11000`. Asimismo, se eligió **coexistencia de Sesiones y JWT**: cookies de sesión en MongoDB para la navegación SSR en Pug y tokens *stateless* `Bearer` (RFC 7519) para la API RESTful `/entradas`.

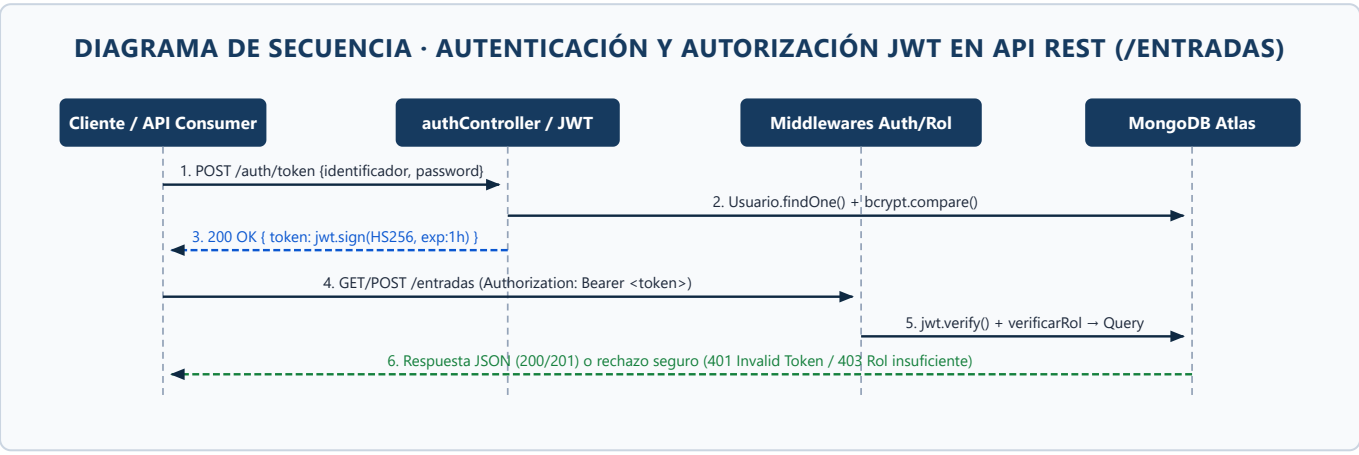
Módulos adicionales incorporados: Se sumaron cinco paquetes justificados por requerimientos concretos del dominio: `helmet` (cabeceras HTTP seguras y Content Security Policy contra XSS y clickjacking), `connect-mongo` (persistencia de sesiones en MongoDB Atlas para sobrevivir reinicios en Render con expiración TTL), `multer` (carga en memoria y validación binaria de fotografías de artistas), `qrcode` (generación en servidor de códigos QR vectoriales en SVG para cada ticket) y `exceljs` (exportación administrativa descargable de pagos y recaudación en formato `.xlsx`).

8. Persistencia de Datos en MongoDB Atlas y Diagramas

La capa de datos se aloja en un clúster cloud de **MongoDB Atlas** e implementa ocho modelos Mongoose (más la colección automática `sesiones` administrada por `connect-mongo`):

Modelo Mongoose	Atributos Principales y Validaciones de Esquema	Relaciones NoSQL y Reglas
Usuario `Usuario.js`	`nombreUsuario` (unique), `email` (unique, regex), `password` (hash bcrypt), `nombre` y `apellido` (solo letras), `dni` (7–8 dígitos), `telefono` (10–15 dígitos), `fechaNacimiento` (+18 años), `rol` (enum 4 roles), `tipoCliente`, `porcentajeDescuento`.	Referenciado por `Transaccion`, `Entrada`, `CancelacionLog` y `Actividad`. Incluye plugin `borradoLogico`.
Sala `Sala.js`	`nombre` (unique), `capacidad` (entero ≥ 1), subdocumento `zonas` (`General`, `Platea`, `VIP` enteros ≥ 0), `ubicacion`, `disponible`. Hook `pre('validate')` recalcula el total.	Relación 1:N con `Evento`. Impide editar zonas o eliminar si posee eventos asociados (HTTP 409).
Entidad `Entidad.js`	`nombre` (unique), `foto` binaria (`Buffer` + `contentType` JPG/PNG/WEBP con `select: false`), `fotoActualizada`, `fotoCredito` (autor y licencia CC).	Relación 1:N con `Evento`. Servida bajo demanda por `GET /media/artistas/:id`.
Evento `Evento.js`	`nombre`, `descripcion`, `categoria` (enum 7 rubros), `fecha`, `hora`, `precio` (General), `precioPlatea`, `precioVIP`, `entradasDisponibles` (min: 0), `vendidosPorZona`.	Referencias `ObjectId` hacia `Sala` y `Entidad`. Métodos atómicos `reservarZona` y `liberarZona`.
Transaccion `Transaccion.js`	`comprobanteId` (unique), `items[]` embebidos (`eventoId`, `tipoEntrada`, `cantidad`, `cantidadCancelada`, `precio`), `subtotal`, `descuentoAplicado`, `total`, `medioPago`, `canal` (`Online` | `Sucursal`).	Referencia a `Usuario` (`cliente`) y a `Evento` en cada ítem. Base de las agregaciones contables.
Entrada `Entrada.js`	`codigoBoleto` (unique `TICK-XXXXXXXXXX`), `sector`, `cantidad`, `precio`, `fila`, `asiento`, `estado` (`Completado` | `Usado` | `Cancelada`), `usadaEn`, `validadaPor`.	Referencias triples hacia `Usuario`, `Evento` y `Transaccion`.
CancelacionLog `CancelacionLog.js`	`nombreCliente`, `eventoNombre`, `codigoBoleto`, `cantidad`, `fechaCancelacion`.	Referencias a `Usuario` y `Evento` para trazabilidad de devoluciones.
Actividad `Actividad.js`	`nombreUsuario`, `rol`, `sesionId`, `tipo` (`sesion` | `navegacion` | `accion` | `click`), `accion`, `pagina`, `metodo`, `estado`, `ip`, `fecha`.	Referencia opcional a `Usuario`. Posee 3 índices compuestos por fecha descendente.





9. Arquitectura MVC, Middlewares y Procesamiento de Peticiones

La aplicación organiza responsabilidades bajo el patrón **Modelo–Vista–Controlador (MVC)**, aislando la configuración de base de datos, las reglas reutilizables de negocio (`utils/`) y once middlewares transversales:

```
├── config/       → config.js (variables .env y secretos) · db.js (MongoDB Atlas + respaldo local)
├── models/       → 8 esquemas Mongoose + plugins/borradoLogico.js
├── controllers/  → 10 controladores (auth, cliente, carrito, admin, eventos, salas, entidades,
entradas, actividad, excel)
├── middlewares/  → authMiddleware, jwt, limiteLogin, origen (CSRF), validaciones, validateSala,
subirFoto, actividad, logger, asyncHandler, errorHandler (+ index.js que los reúne)
├── routes/       → 9 enrutadores Express modulares montados en index.js
├── utils/        → transaccion.js (ACID/compensación), paginar.js, qr.js, excel.js, escapeRegex.js
└── views/        → Plantillas Pug (layouts Admin/Cliente/Acceso, partials reutilizables y vistas por
rol)
```

Ruta / Prefijo y Método	Cadena de Middlewares y Comportamiento del Controlador
`POST /auth/login` `GET /auth/logout`	`limiteLogin` → `procesarLogin` verifica hash `bcrypt`, regenera sesión en MongoDB y redirige según rol. `/logout` destruye la sesión y limpia la cookie `connect.sid`.

Ruta / Prefijo y Método	Cadena de Middlewares y Comportamiento del Controlador
`POST /auth/token`	`limiteLogin` → `emitirToken` valida credenciales y firma un **JWT** (HS256, audiencia, emisor y expiración `1h`) sin exponer datos sensibles en el payload.
`GET|POST|PUT|DELETE /entradas`	API JSON: `autenticarToken` → `verificarAutenticacion` → `verificarRol(['Administrador'])` (en escritura). Lista tickets propios o emite/anula entradas ajustando stock.
`GET /cliente/comprar` `POST /cliente/carrito/procesar-pago`	Cartelera pública con filtros combinados. El pago requiere sesión activa, aplica descuento de membresía y ejecuta `conTransaccion()` reservando zona y emitiendo tickets con QR.
`POST /cliente/cancelar-entrada/:id`	`verificarRol(['Cliente'])` → `Entrada.cancelarEntradaZona` cambia estado a `Cancelada`, devuelve el cupo al sector, descuenta de `Transaccion` y graba `CancelacionLog`.
`GET|POST /admin/boleteria` `GET|POST /admin/validar/:codigo`	`verificarRol(['Administrador','Supervisor'])`. Venta presencial en sucursal con ticket imprimible, y escaneo de QR en puerta con marcado atómico `Usado`.
`GET /admin/informe` · `/pagos` `GET /admin/informe/excel`	`verificarRol(['Administrador','Supervisor','Lector'])`. Pipelines de agregación MongoDB sobre recaudación neta y descarga directa en formato `.xlsx`.

10. Trazabilidad de Requerimientos (RF / RNF) y Documentación de Pruebas

Integrando los artefactos de *Ingeniería de Software*, se detalla la trazabilidad de requerimientos clave y la matriz de pruebas funcionales y de seguridad ejecutadas sobre el sistema desplegado:

Código	Descripción del Requerimiento	Implementación Técnica Verificada
RF-01 / RF-09	Gestión de salas por sectores, artistas con foto y eventos con cupo controlado	Modelos `Sala`, `Entidad` y `Evento` con validación de suma de zonas, carga binaria con `multer` y control de capacidad máxima.
RF-06 / RF-12	Venta online (con membresías) y venta presencial en boletería física	Controladores `carritoController` y `adminController.procesarVentaFisica` unificados bajo `Transaccion` (canal `Online` / `Sucursal`).
RF-07 / RF-14	Emisión de entradas con QR y validación única en puerta	Generación SVG con `utils/qr.js` y validación atómica en `POST /admin/validar/:codigo` que impide doble acreditación.
RF-15 / RF-16	Cancelación con devolución de stock y reportes gerenciales exportables	Método transaccional `cancelarEntradaZona`, agregaciones netas en `entradasController` y exportación con `excelController`.
RNF-01	Seguridad de acceso, sesiones, API stateless y protección de cabeceras	Hash `bcrypt`, cookies seguras, JWT HS256, `helmet` (CSP), anti-CSRF (`verificarOrigen`) y rate-limit (`limiteLogin`).

Código	Descripción del Requerimiento	Implementación Técnica Verificada
RNF-02	Integridad transaccional y consistencia ante compras concurrentes	Filtros condicionales atómicos (`$gte` / `$inc`) en MongoDB y wrapper `conTransaccion()` para todo el flujo de compra/cancelación.
RNF-03	Rendimiento y escalabilidad en auditoría de alto volumen	Consultas `lean()` paginadas, acotación temporal e índices compuestos en `Actividad` (respuesta reducida de ~1,4 s a ~0,4 s con 200k registros).

Matriz de Pruebas Funcionales, de Seguridad y Casos Límite

ID	Objetivo de la Prueba	Procedimiento Ejecutado	Resultado Obtenido
CP-01	Validación multicapa en registro (+18 años, DNI y email)	Envío de `POST /auth/registro` omitiendo validaciones HTML con DNI inválido o fecha de menor de 18 años.	**Exitoso:** Middleware y esquema rechazan con HTTP 400 mostrando alerta precisa sin persistir en BD.
CP-02	Bloqueo por fuerza bruta en login web y API	Ejecución de 6 peticiones consecutivas con clave errónea hacia `POST /auth/login` y `POST /auth/token` .	**Exitoso:** Al 6° intento responde `429 Too Many Requests` bloqueando temporalmente por 15 min.
CP-03	Autenticación JWT y rechazo de tokens inválidos/vencidos	Obtención de token en `/auth/token` y consumo de `GET /entradas` con token válido vs. token adulterado.	**Exitoso:** Con token válido retorna `200 OK` ; adulterado o vencido devuelve `401` + header `WWW-Authenticate` .
CP-04	Control estricto de roles y protección de Superadmin	Intento de acceso con usuario `Lector` o `Cliente` a rutas de edición `/admin/salas` o intento de borrar a `admin` .	**Exitoso:** Responde página/JSON `403 Acceso Denegado` ; el usuario `admin` permanece protegido en servidor.
CP-05	Compra online con descuento VIP y control de sobregiro	Compra con `cliente1` (VIP 30 %) superando y luego respetando el cupo disponible del sector VIP.	**Exitoso:** Rechaza exceso de cupo; con stock válido aplica 30 % OFF, crea `Transaccion` y emite QR.
CP-06	Validación única de QR y bloqueo de cancelación post-uso	Acreditación de ticket en `POST /admin/validar/:codigo` , segundo escaneo e intento posterior de cancelación.	**Exitoso:** Primer escaneo marca `Usado` ; el reescaneo alerta ticket ya utilizado y cancelar devuelve HTTP 409.
CP-07	Integridad referencial y borrado lógico	Intento de eliminar una sala con eventos activos y posterior eliminación de un evento sin ventas conflictivas.	**Exitoso:** Sala con eventos devuelve HTTP 409; la baja marca `eliminado: true` conservando el histórico.

11. Mantenimiento, Evolución y Uso Consciente de Inteligencia Artificial

Mantenimiento correctivo y evolutivo: Respecto a la segunda entrega, se refactorizó por completo la capa de autenticación reemplazando el almacenamiento de sesiones en memoria (que perdía sesiones ante cada reinicio) por `connect-mongo` con expiración automática, y configurando `app.set('trust proxy', 1)` junto a una política de `Referrer-Policy` compatible para que las cookies seguras HTTPS y la verificación CSRF operen correctamente detrás del balanceador de Render. Para mejorar la **escalabilidad y el rendimiento**, se reemplazaron cálculos manuales en JavaScript por pipelines de agregación en MongoDB para los informes financieros, se ocultaron los buffers binarios de imágenes en listados generales mediante `select: false` y se incorporaron índices compuestos y paginación en la colección `Actividad`.

Declaración sobre el uso de Inteligencia Artificial: Siguiendo los lineamientos institucionales de la cátedra, el equipo utilizó herramientas de IA generativa como asistente de consulta y apoyo productivo en tres tareas acotadas: (1) normalización de hojas de estilo CSS e indentación de mixins reutilizables en plantillas Pug complejas, (2) revisión preventiva de casos borde de seguridad web (directivas de `helmet` y expresiones regulares de validación) y (3) revisión ortotipográfica de la documentación técnica y de la API (`docs/API.md`). Todo fragmento sugerido fue analizado críticamente, adaptado a las convenciones del proyecto, probado sobre MongoDB Atlas y validado por los integrantes responsables de cada módulo para garantizar su total comprensión y dominio técnico.

12. Conclusión Final del Cuatrimestre

- ✓ **¿Qué aprendimos este cuatrimestre?** Comprendimos cómo estructurar profesionalmente un servidor web en Node.js y Express aplicando separación de responsabilidades MVC, manejo centralizado de errores, asincronía con `async/await`, modelado documental en MongoDB Atlas y esquemas reales de seguridad (diferenciando cuándo aplicar sesiones de servidor con cookies seguras y cuándo utilizar autenticación *stateless* mediante JSON Web Tokens), llevando finalmente el producto a un entorno real en la nube.
- ✓ **¿Qué dificultades tuvimos y cómo las resolvimos?** Los principales desafíos fueron tres: (1) compatibilizar las transacciones ACID entre MongoDB Atlas (clúster con replica set) y entornos locales *standalone* de desarrollo, lo cual resolvimos diseñando el utilitario `utils/transaccion.js` que detecta la topología al iniciar y aplica compensaciones automáticas en local; (2) lograr que las cookies seguras `secure: true` y la protección CSRF funcionaran detrás del proxy HTTPS en la nube ajustando `trust proxy` y las cabeceras de `helmet`; y (3) evitar que el crecimiento de la bitácora de auditoría degradara los tiempos de carga, resuelto con índices compuestos y consultas acotadas con `lean()`.
- ✓ **¿Qué parte del desarrollo Back End nos interesó más?** Nos resultó especialmente atractiva la arquitectura de seguridad y control de accesos por middlewares (combinar autenticación, roles jerárquicos, firma de tokens JWT y validaciones atómicas en la base de datos), así como ver reflejadas las reglas de negocio del mundo real —como evitar que dos compras simultáneas sobrevendan una sala o que un QR se utilice dos veces en la puerta— directamente en operaciones seguras del servidor.
- ✓ **¿Qué deberíamos reforzar a futuro?** De cara al Proyecto Final de la carrera, buscamos profundizar en la escritura de suites de pruebas automatizadas de integración extremo a extremo (E2E), la integración con pasarelas de pago y APIs externas en tiempo real mediante WebSockets, y la automatización de pipelines CI/CD para despliegues continuos.